Enterprise Agentic AI Banking Platform

Engineering Handbook

Version 1.0 (Frozen)

---

1. Purpose

This handbook is the single source of truth for implementing the Enterprise Agentic AI Banking Platform Version 1.0.

The objective is to build one complete, executable business journey from AI Agent to Payment Service. The architecture is frozen. Future implementation must follow this handbook exactly.

---

2. Version 1.0 Goal

Implement one complete payment journey.

```
Enterprise Agent
        │
        ▼
Enterprise MCP Server
        │
        ▼
Workflow Runtime (Temporal)
        │
        ▼
Payment MCP Server
        │
        ▼
Payment Service
        │
        ▼
H2 Database
```

Only after this flow works end-to-end will additional banking capabilities such as Treasury, Tax, FX, Customer, and Limits be added.

---

3. Repository Landscape

```
enterprise-agentic-ai-banking-platform

common-domain
common-contract

payment-service
payment-mcp-server

workflow-runtime

enterprise-mcp-server

enterprise-agent
```

---

4. Repository Build Order

Repositories shall always be implemented in the following order.

1. common-domain
2. common-contract
3. payment-service
4. payment-mcp-server
5. workflow-runtime
6. enterprise-mcp-server
7. enterprise-agent

Do not change this order.

---

5. Repository Responsibilities

common-domain

Contains shared domain models.

Examples:

- Enums
- Common constants
- Domain objects
- Shared value objects

No Spring Boot.

common-contract

Contains API contracts.

Examples:

- Request DTO
- Response DTO
- Error DTO

No business logic.

---

payment-service

Owns the Payment business capability.

Responsibilities

- Payment orchestration
- Account debit
- Account credit
- Payment persistence

Technology

- Spring Boot
- Spring Data JPA
- H2

---

payment-mcp-server

Exposes Payment capability through MCP.

Responsibilities

- MCP Tool Registration
- Request Mapping
- Response Mapping

No business logic.

---

workflow-runtime

Owns all Temporal workflows.

Responsibilities

- Workflow orchestration
- Activities
- Retry
- Compensation
- Saga

Business services must never know Temporal.

---

enterprise-mcp-server

Enterprise orchestration layer.

Responsibilities

- Register multiple MCP Servers
- Tool discovery
- Capability routing

No banking business logic.

---

enterprise-agent

LLM entry point.

Responsibilities

- Prompt
- Planning
- Tool selection
- Tool invocation

Business services remain unaware of AI.

---

6. Technology Stack

Java 17

Spring Boot 3.5.x

Spring Data JPA

Spring Validation

H2 Database

Temporal Java SDK

Spring AI

Maven

Lombok

---

7. Standard Package Structure

Every Spring Boot service shall use the same package layout.

```
api

service

repository

entity

configuration

exception

util
```

No additional package structures unless approved.

---

8. Service Standards

Services represent business capabilities.

Examples

- PaymentService
- AccountService
- TreasuryService
- TaxService

Never create technical services such as

- HelperService
- ExecutionService
- PersistenceService
- UtilityService

- ManagerService

---

9. Repository Standards

Repositories perform persistence only.

Repositories contain no business logic.

Use Spring Data JPA.

---

10. Validation Standards

Layer 1

Bean Validation

Examples

- @NotNull
- @NotBlank
- @Positive

Layer 2

Business validation inside business services.

---

11. Aggregate Ownership

Payment Aggregate

Owned by PaymentService

Account Aggregate

Owned by AccountService

Services shall not directly manipulate another aggregate.

---

12. Controller Standards

Controller

↓

Service

↓

Repository

No business logic in controllers.

---

13. Transaction Standards

Transactions begin in the service layer.

Use @Transactional.

Repositories are not transactional boundaries.

---

14. Exception Standards

Base

BusinessException

Derived

PaymentValidationException

Use one GlobalExceptionHandler.

---

15. Dependency Direction

```
enterprise-agent

↓

enterprise-mcp-server

↓

workflow-runtime

↓
```

```
payment-mcp-server

↓

payment-service
```

Reverse dependencies are not permitted.

---

16. Coding Standards

Constructor injection only.

Use @RequiredArgsConstructor.

Never use field injection.

Never use setter injection.

---

17. Persistence Standards

Version 1.0

Spring Data JPA

↓

H2 Database

Future database migration must require configuration changes only.

---

18. Development Standards

Every repository must

- compile independently
- start independently
- expose a health endpoint
- include application.yml
- include schema.sql
- include data.sql

---

19. Repository Delivery Process

Implementation

↓

Review

↓

Repository Regeneration

↓

Compile

↓

ZIP

↓

Freeze

Each repository receives its own version.

Example

payment-service-v1.0.zip

Future corrections

payment-service-v1.1.zip

---

20. Non-Negotiable Rules

1. Architecture Version 1.0 is frozen.
2. No redesign during implementation.
3. No placeholder implementations.
4. No fake code.
5. No stubs.
6. No unnecessary abstractions.
7. One repository at a time.
8. Every repository must compile independently.
9. Repository ZIP is the delivery artifact.
10. Business services must not depend on MCP, Temporal, or AI.

11. Package structure is fixed.
12. Technology stack is fixed.
13. Naming conventions are fixed.
14. Changes after release create a new repository version.

---

21. Definition of Done

Version 1.0 is complete only when the following flow executes successfully.

```
Enterprise Agent

↓

Enterprise MCP

↓

Workflow Runtime

↓

Payment MCP

↓

Payment Service

↓

H2 Database

↓

Payment Success Response
```

This handbook is the engineering contract for all future implementation work.